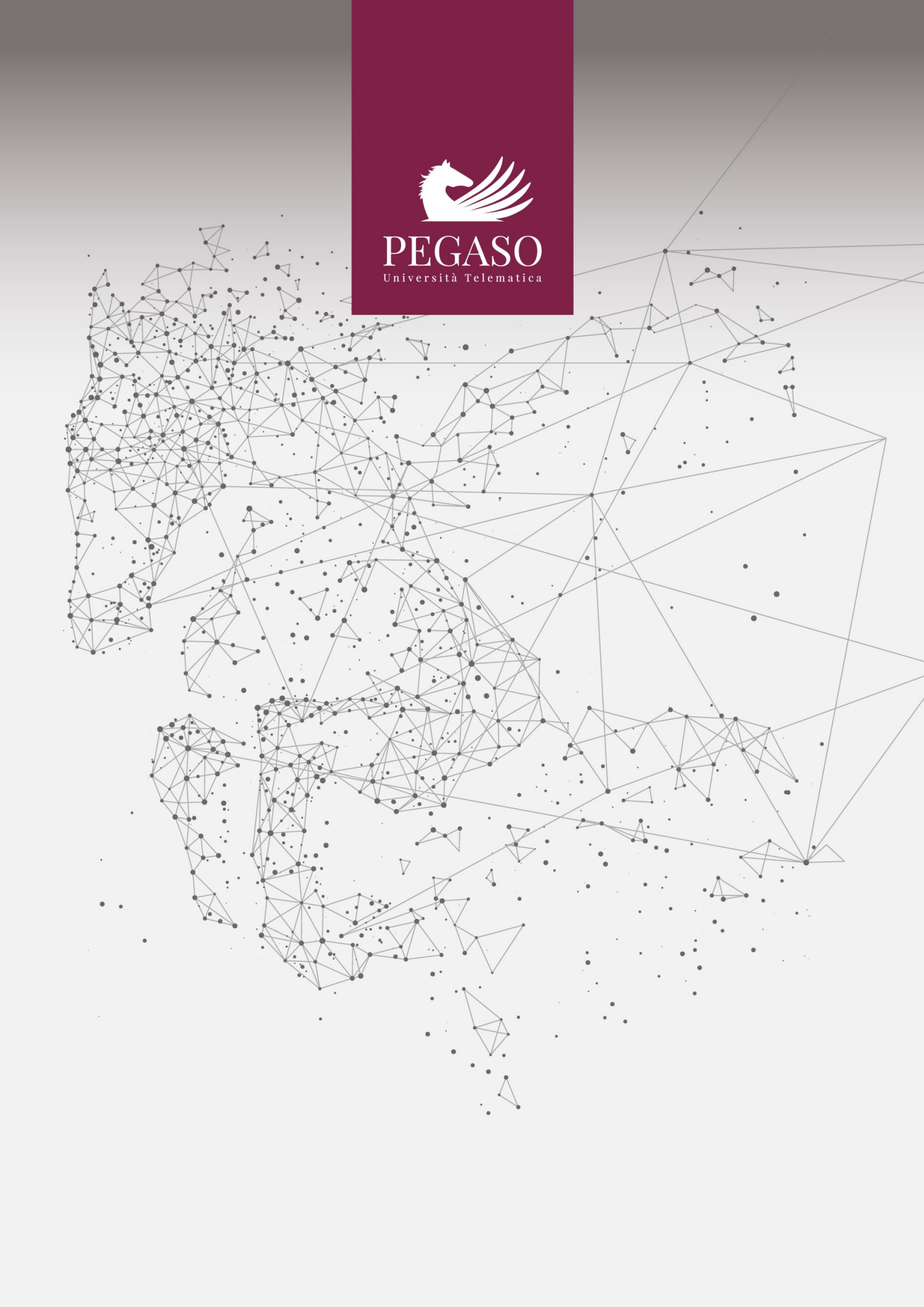

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. INTRODUZIONE AD ANACONDA 4
3. INTRODUZIONE ALLA LIBRERIA NUMPY 9
4. INTRODUZIONE AGLI ARRAY IN NUMPY 11
BIBLIOGRAFIA 17

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta Anaconda una delle piattaforme sviluppate per Python per la Data Science e i Big Data che conta oltre 1.500 package software già installati. Viene, inoltre, presentato un nuovo ambiente per la scrittura di codice Jupyter Notebook, basato per l'appunto su notebook, un applicativo web che permette di scrivere un intero flusso di lavoro in verticale. Successivamente, viene presentata la libreria NumPy che offre un insieme molto ricco di funzionalità matematiche ed è basata sul concetto di array. Infine, si illustrano le funzioni di base per la definizione di array Numpy.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Introduzione ad Anaconda

Finora, per la programmazione in Python è stato utilizzato l'IDLE di Python. Tale ambiente, però, rappresenta delle limitazioni nel momento in cui si iniziano ad utilizzare alcune librerie o gli strumenti per la Data Science e i Big Data.

Uno strumento fondamentale e completo è rappresentato dalla piattaforma **Anaconda**, una distribuzione dei linguaggi di programmazione Python e R per l'informatica scientifica (data science, applicazioni di machine learning, elaborazione dati su larga scala, analisi predittiva, ecc.) che mira a semplificare la gestione e la distribuzione dei pacchetti.

La distribuzione include pacchetti di data science adatti per Windows, Linux e macOS. La versione gratuita di Anaconda per scopi accademici è l'Anaconda Individual Edition che conta oltre 1.500 package software, tra cui:

- package manager **Conda;**
- ambienti per la scrittura di codice come **IPython, Jupyter e Spyder**;
- strumenti per la Data Science come **Pandas, Numpy e Numba**;
- librerie per il **Machine Learning**;
- tool per la visualizzazione dei risultati (es. **Matplotlib**).

Anaconda include anche una GUI (Graphical User Interface) denominata Anaconda Navigator come alternativa grafica all'interfaccia della riga di comando (CLI).

Nella figura successiva si riporta il logo di Anaconda (a sinistra) e i loghi di alcuni package contenuti nella distribuzione (a destra).

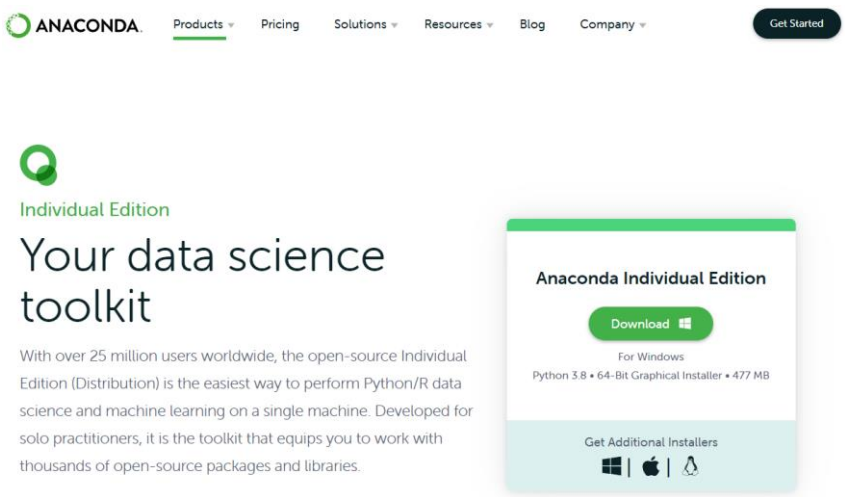


PEGASO
Università Telematica

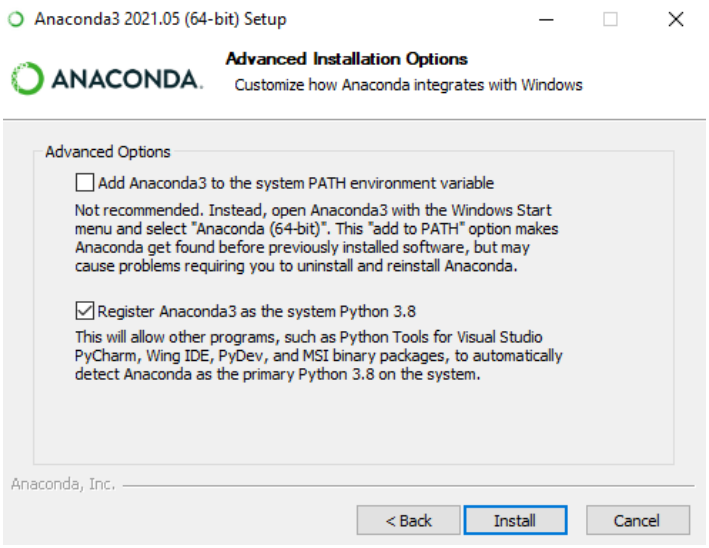
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Procediamo con l'installazione di Anaconda Individual Edition disponibile all'indirizzo:

https://www.anaconda.com/products/individual

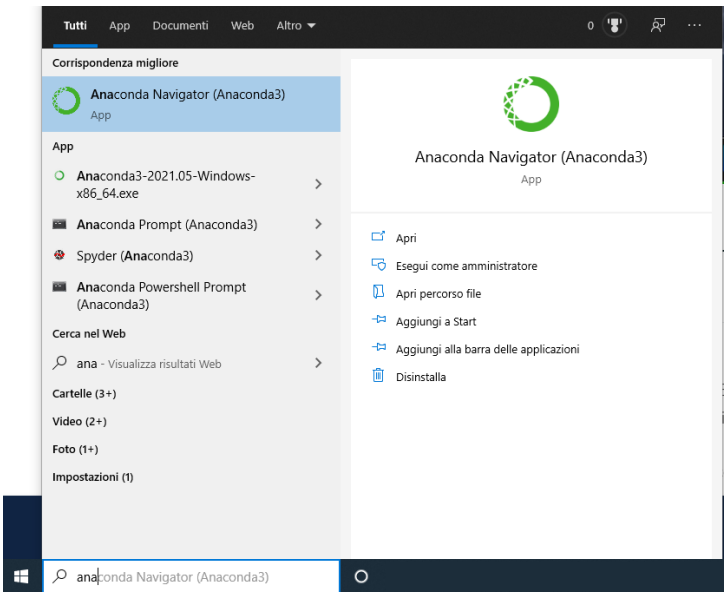


Dopo il download e doppio click per far partire l'installazione, nella schermata "Advanced Installation Options" selezionare solo il secondo checkbox, come indicato nella figura seguente:

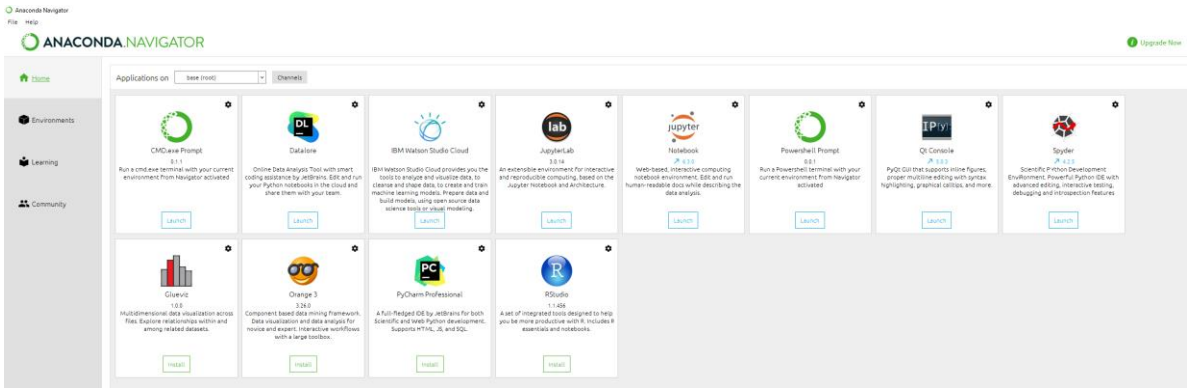


Al termine dell'installazione, per far partire il Navigator di Anaconda, cliccare su Start di Windows e digitare Anaconda Navigator come indicato nella figura seguente.

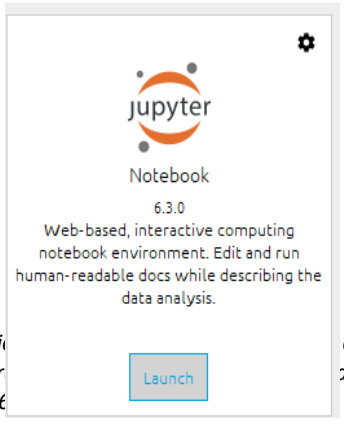
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Il Navigator è un grande pannello in cui esiste un comando per ogni strumento invocabile attraverso Anaconda (come indicato nella figura successiva).



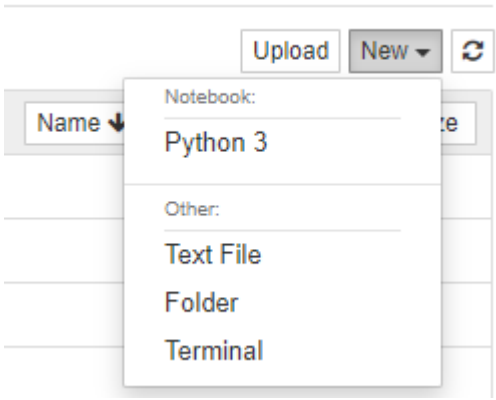
Nella parte centrale si può notare che c'è un pulsante che riporta l'etichetta **LAUNCH** se lo strumento è disponibile oppure **INSTALL** se non è ancora presente nell'ambiente di lavoro. Procediamo con il Launch di Jupyter Notebook già installato.



Attenzione! Questo materi dello studente ed è coperto da copyright. Ne è severamente vietata la ripr ale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 6

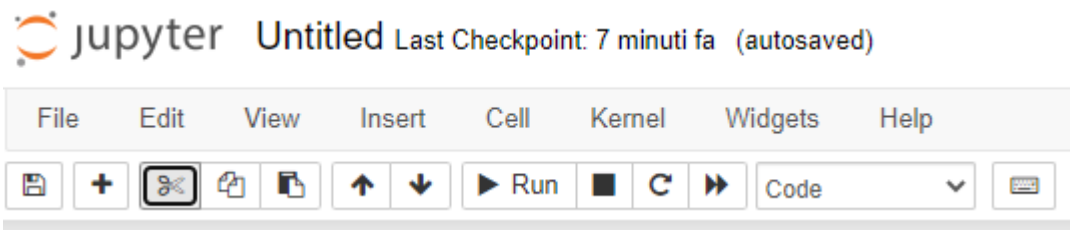
Ci verrà chiesto con che browser vogliamo aprire Jupyter Notebook. Nel caso presentato, il browser predefinito sarà Chrome.

Navigando opportunamente nel file system è possibile scegliere il percorso in cui verranno salvati gli script. È possibile (attraverso il pulsante New) creare, inoltre, nuove cartelle o nuovi script Python 3.



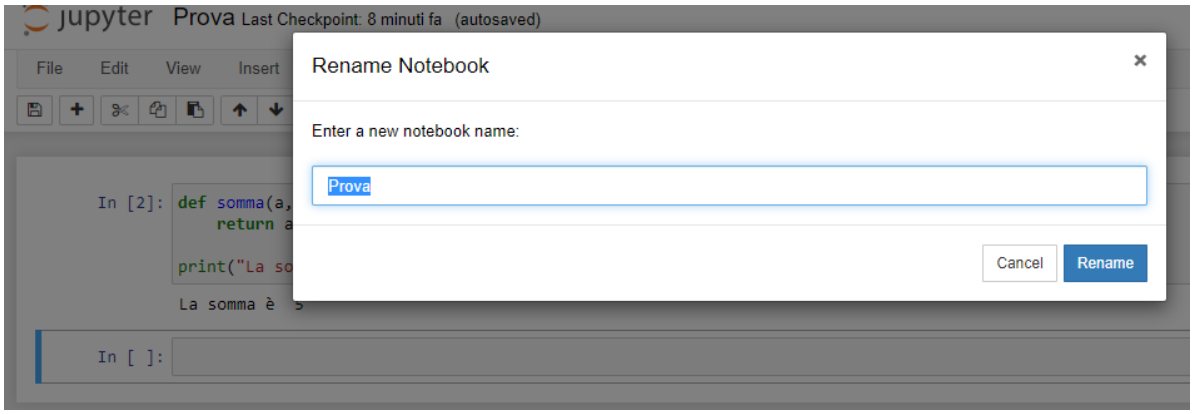
Un **notebook** (di Jupyter) è un applicativo web che permette di scrivere un intero flusso di lavoro in verticale. È costituito da blocchi detti **celle** ognuno dei quali può ospitare codice (o anche immagini). I notebook possono essere usati con vari linguaggi e possono essere esportati in vari modi, tra cui pdf. Tutto il lavoro viene salvato in un unico file con estensione .ipynb. La barra del menu del notebook in alto (si veda la Figura successiva) contiene:

- menu **File** con i comandi per la gestione del notebook (creazione, salvataggio, esportazione e stampa);
- menu **Edit** per la modifica delle singole celle (copia, incolla, cancellazione, spostamento);
- menu **View** per abilitare/disabilitare la visualizzazione di elementi quali la Toolbar (con icone) e l'intestazione in alto con logo e titolo;
- menu **Insert** per l'inserimento di una cella nel punto desiderato;
- menu **Cell** per gestire l'esecuzione di una o più celle.



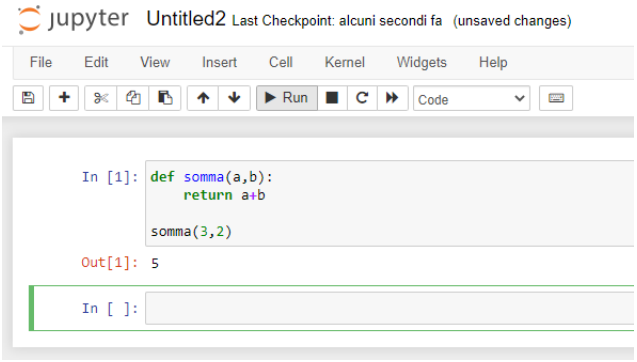
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per modificare il nome del file, basta cliccare sul nome del notebook (solitamente “Untitled”) e apparirà la schermata di modifica come indicato nella figura seguente.



Per mostrare l’utilizzo di uno Jupyter notebook, scriviamo un semplice esercizio con la funzione somma() come indicato nella figura seguente.

Si noti che in questo esempio non è stato richiamata la funzione print() poiché viene visualizzato a schermo il valore restituito dalla funzione, o meglio l’ultimo valore restituito dalla cella. Ovviamente la funzione print() può essere richiamata ovunque nel codice sorgente e quante volte si desidera.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d’autore (L. 22.04.1941/n. 633).

3. Introduzione alla libreria NumPy

NumPy è una delle librerie più importanti di Python in quanto offre un insieme ricchissimo di funzionalità matematiche. Alcune operazioni eseguibili con NumPy sono eseguibili anche con le funzionalità offerte dal linguaggio Python. La differenza principale è che con NumPy le **prestazioni** e la **produttività** sono superiori.

Il funzionamento di NumPy si basa su una struttura dati fondamentale di tipo n-dimensionale l'**array**. Avendo installato la piattaforma Anaconda abbiamo già a disposizione la libreria NumPy. Come già visto in Python per le liste, anche gli array possono essere:

- **monodimensionali** (una sequenza di elementi);
- **multidimensionali** (dati disposti su più righe) come le matrici matematicamente di due più dimensioni.

Come visto finora, per utilizzare la libreria NumPy, essa dovrà essere importata. Per semplificare il richiamo della libreria per la chiamata delle funzioni in essa contenute è possibile utilizzare la keyword **as** per la creazione di un **alias.** L'istruzione

import numpy as np

permette, quindi, di far riferimento alla libreria, semplicemente usando il termine **np**.

Procediamo, dunque, con la creazione del primo array a partire da una lista di numeri (che verrà passata come parametro) con la funzione **array**() di numpy come indicato di seguito.

```
import numpy as np
primo_array = np.array([1,2,3,4,5,6])
print(primo_array)
```

```
[1 2 3 4 5 6]
```

Gli array differiscono dalla lista per due motivi:

- gli array sono **omogenei** (tutti gli elementi dello stesso tipo);
- come è possibile vedere dalla visualizzazione a schermo, **non ci sono le virgole** utilizzate come separatori tra gli elementi numerici.

Andiamo ad analizzare il tipo di dato per l'array *primo_array* con la funzione type(). Il risultato sarà il tipo ndarray della libreria numpy.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
type(primo_array)
```

```
numpy.ndarray
```

Quando NumPy crea un array **non copia i dati dall'origine** al nuovo array, ma crea solo un **collegamento** per motivi di efficienza. Questo significa che, per default, un array NumPy è un po' come una **vista** sui dati sottostanti non una loro copia. Se cambia l'oggetto contenente i dati sottostanti cambiano anche i dati dell'array.

Se il collegamento ai dati non fosse il comportamento desiderato, basterà passare al costruttore il parametro **copy=True**.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Introduzione agli array in NumPy

Per mostrare le funzionalità di NumPy, inizieremo con la definizione di alcuni array di esempio sfruttando anche alcune funzioni della libreria stessa.

Definiamo un array con i primi 10 numeri interi positivi utilizzando la funzione range() e con la copia attiva.

```
import numpy as np
numeri = np.array(range(1,11), copy = True)
print(numeri)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

Ora definiamo un array con gli elementi tutti ad uno (di default sono numeri in virgola mobile) con la funzione **ones**() che prende come argomento la dimensione dell'array.

```
import numpy as np
numeri = np.ones(5)
print(numeri)
```

```
[1. 1. 1. 1. 1.]
```

Procediamo definendo un array con gli elementi tutti a zero (di default sono numeri in virgola mobile) con la funzione **zeros**() che prende come argomento la dimensione dell'array.

```
import numpy as np
numeri = np.zeros(5)
print(numeri)
```

```
[0. 0. 0. 0. 0.]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione array() può ricevere come argomento per la creazione di un array:

- **lista;**
- **array;**
- **tupla.**

Numpy determina in maniera automatica il tipo degli elementi dell'argomento passato come parametro, a meno che non venga esplicitato con il parametro **dtype**. I tipi supportati da NumPy sono una ventina, tra cui bool (booleano), int64 (intero su 64 bit), uint64 (intero senza segno su 64 bit), float64 (numero in virgola mobile su 64 bit), U32 (intero senza segno su 32 bit).

Definiamo un array di tutti elementi uguali ad 1, ma, differentemente dal default, tutti di tipo intero esplicitando *dtype = np.uint64*.

```
import numpy as np
numeri = np.ones(5, dtype = np.uint64)
print(numeri)
```

```
[1 1 1 1 1]
```

Un aspetto molto interessante di NumPy è l'uso con le **matrici**. Le matrici possono essere create come liste di liste (liste bidimensionali). La visualizzazione delle matrici risulta essere chiara direttamente utilizzando la funzione print() come si può osservare nella figura seguente.

```
import numpy as np
matrice = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrice)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

È possibile inizializzare delle matrici anche con la funzione ones() o zeros() a cui verrà passata una lista composta dalle dimensioni della matrice (nell'esempio che segue una matrice 5x3 – 5 righe e 3 colonne - di valori tutti uguali a 1.0).

```
import numpy as np
numeri = np.ones([5,3])
print(numeri)
```

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

Per ogni array NumPy memorizza:

- il numero di dimensioni nell'attributo **ndim** (ndim = len(shape));
- la forma nell'attributo **shape;**
- il tipo di dati nell'attributo **dtype.**

Nell'esempio che segue, ndim sarà uguale a 2 (la matrice, infatti, ha due dimensioni), la forma sarà 5x3 e il dtype sarà quello di default float64.

```
import numpy as np
numeri = np.ones([5,3])
print("ndim = ",numeri.ndim)
print("shape = ",numeri.shape)
print("dtype = ",numeri.dtype)
```

```
ndim =  2
shape =  (5, 3)
dtype =  float64
```

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per chiarezza, visualizziamo l'attributo **shape** prima su un array monodimensionale e, successivamente su una matrice. Come è possibile osservare dalla figura successiva, il risultato sarà sempre una tupla che:

- nel primo caso avrà valore 6, (una sola dimensione e 6 elementi);

nel secondo caso avrà valore 3,3 (due dimensioni e 3x3 elementi).

```
import numpy as np
primo_array = np.array([1,2,3,4,5,6])
matrice = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(primo_array.shape)
print(matrice.shape)
```

```
(6,)
(3, 3)
```

Per le matrici Numpy offre anche la funzione **eye**() con il seguente formato:

eye(N, M = None, k=0, dtype=np.float)

che definisce una matrice NxM:

- costituita da una serie di "1" nella diagonale k-esima;
- in cui tutti gli altri elementi al di fuori della diagonale sono pari a "0";
- per cui il valore k positivo rappresenta il numero di diagonali sopra la diagonale principale.

Inoltre, se **M = None** allora M=N. Di seguito si riporta un esempio di una matrice quadrata 3x3 in cui tutti gli elementi della diagonale principale sono uguali ad 1.

```
import numpy as np
numeri = np.eye(3)
print(numeri)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta un esempio di una matrice quadrata 3x3 in cui tutti gli elementi della diagonale al di sopra di quella principale (k=1) sono uguali ad 1.

```
import numpy as np
numeri = np.eye(3, k=1)
print(numeri)
```

```
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 0.]]
```

Di seguito si riporta un esempio di una matrice quadrata 3x5 in cui tutti gli elementi della diagonale al di sopra di quella principale (k=1) sono uguali ad 1. Anche se è improprio parlare di diagonale principale in una matrice non quadrata, il concetto di diagonale è facilmente comprensibile ed adattabile anche per le matrici rettangolari.

```
import numpy as np
numeri = np.eye(3, M=5, k=1)
print(numeri)
```

```
[[0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]]
```

Nel caso in cui fosse necessario moltiplicare più matrici, si usa la matrice identità eye(N) quale valore iniziale dell'accumulatore della catena delle moltiplicazioni.

NumPy oltre alla funzione range() offre modi più efficienti per generare array di numeri ben intervallati attraverso la funzione **arange**() che ha il seguente formato:

arange([start,] stop[, step,], dtype = None)

in cui **start** (opzionale, se non indicato start=0) è il valore di partenza incluso, **stop** (obbligatorio) è il valore di fine escluso, **step** (opzionale, se non indicato step=1) è l'incremento per ottenere i numeri dell'intervallo e **dtype** è il tipo di dato degli elementi dell'array.

```
import numpy as np
numeri = np.arange(2, 5, 0.25)
print(numeri)
```

```
[2.   2.25 2.5  2.75 3.   3.25 3.5  3.75 4.   4.25 4.5  4.75]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione arrange() serve anche per creare array in ordine decrescente come indicato nel seguente caso in cui start>stop e step<0.

```
import numpy as np
numeri = np.arange(5, 2, -0.25)
print(numeri)
```

```
[5.   4.75 4.5  4.25 4.   3.75 3.5  3.25 3.   2.75 2.5  2.25]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev, D. (2017). Data science con Python. Apogeo.
- Maggi, G. (2020). Data science con Python. Edizioni LSWR.
- Marin, I., Shukla, A. Sarang, VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).